



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4305 · Software Engineering · Fall 2026

Agile Development I — The Manifesto, User Stories and Scrum

Why agile is an argument about cost, what a sprint actually consumes, and how four students run a fortnight

Session 5 of 28 · Tuesday 22 September 2026 · 1:00 – 2:20 PM

· Prof. Guillaume Sanje Mpacko, Ph.D.

Agile Development I — The Manifesto, User Stories and Scrum

SESSION 5 OF 28

Tuesday, 22 September 2026 · 1:00 – 2:20 PM · Unit 1 · Foundations and Process

LEARNING OBJECTIVES

- ✓ State the four values of the Agile Manifesto, including the clause that is almost always dropped, and name the economic condition that makes each trade rational.
- ✓ Write a well-formed user story with acceptance criteria, and test it against INVEST.
- ✓ Explain what a story point is, and how points become a team's velocity.
- ✓ Name Scrum's three accountabilities, three artifacts and five events, and say for each one *who owns it* and *what it inspects*.
- ✓ Distinguish the product backlog from the sprint backlog, and build both for SANTE-BASSAM.

FROM SESSION 4

We finished by drawing a line through SANTE-BASSAM. Above it sat the things that must be specified once and got right the first time — the patient identifier format, the consent text, the sync contract. Below it sat everything the clerk, the community health worker and the nurse have to teach us by using the software.

Sessions 3 and 4 gave you the machinery for above the line. Today and Friday supply the machinery for below it.

TEAM PROJECT · 0

first 15 minutes of class

Sprint 0 review — S-01, S-02, S-03

From now on every session opens with fifteen minutes on SANTE-BASSAM, applying what the previous session taught. Today is the exception that proves the rule: we start before the vocabulary arrives, on purpose.

WHAT HAPPENS

Three things were built in sprint 0. Split into your three pairs, take one each, and spend five minutes answering the questions below **in ordinary language** — no jargon, no template. Then two minutes each to report to the other four.

Pair A — S-01

the journaled local store on both devices

Pair B — S-02

locally generated patient identifiers

Pair C — S-03

the paper register, migrated

1. **Who wanted this, and why?** Name a person — the clerk, the CHW, the nurse, the officer. “The system” is not a person.
2. **How would you show the nurse it works**, without showing her any code?
3. **What would have broken** if it had not been built?

Keep what you wrote. In §3 you meet the template those three answers were secretly asking for, and your first homework question is to hold your own sentence up against it.

Recorded in: the team progression log, one row per session, kept by the current Scrum Master. Both names of each pair go against the story they took.

1 · The one question that decides the method

In Session 3, the waterfall team met the clerk in week 12. That was not carelessness. It was the correct decision *given the cost of the alternative*. In 1970, putting working software in front of a user meant machine time, punched cards and an operator — so the cheapest way to get a requirement right was to think very hard before writing any code.

That cost has collapsed. Everything that people call “agile” follows from the collapse. So before adopting any method, ask one question:

THE QUESTION

How much does it cost to put something that works in front of a real user, and to act on what they say?

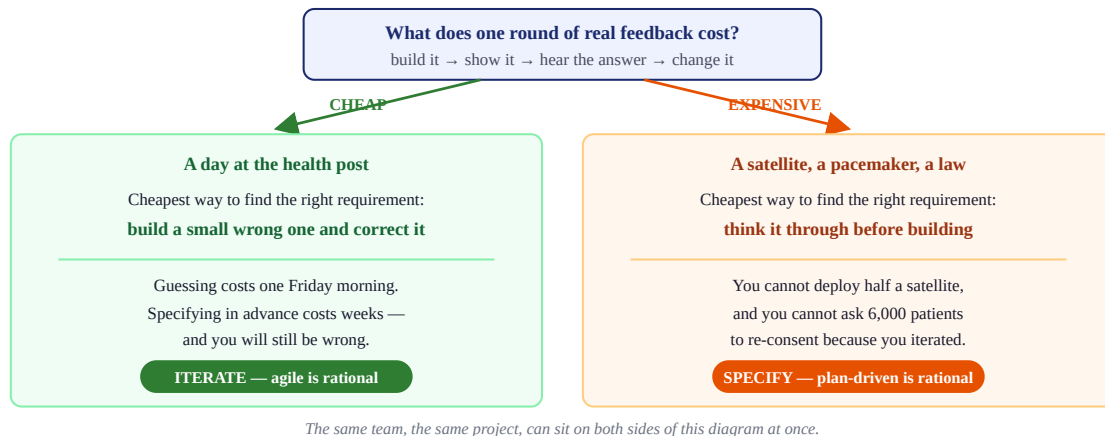


Figure 1. Agile is not newer, faster or more modern. It is the rational answer to one question about cost. Where the answer changes, the correct method changes with it — which is why SANTE-BASSAM will be run both ways at the same time.

APPLIED TO SANTE-BASSAM

Price the feedback loop before choosing the method

Cheap — the workflow

The post is twenty minutes away. The clerk will sit with you on a Friday morning. One round of feedback on the registration screen costs one morning and one bus fare. **Iterate.**

Expensive — the law and the contract

The patient identifier format, the consent wording and the sync contract are used by 6,000 records and a second device. Getting them wrong costs a migration, or a legal problem. **Specify once.**

2 · The Agile Manifesto, in full

In February 2001, seventeen practitioners who had each been running projects in a lighter way met at a ski lodge in Snowbird, Utah. They disagreed about almost everything except four sentences. Those four sentences are the whole of the Manifesto, and they are short enough to read in class. Read them before anyone tells you what they mean.

MANIFESTO FOR AGILE SOFTWARE DEVELOPMENT — THE FOUR VALUES, VERBATIM

Individuals and interactions

OVER

processes and tools

Working software

OVER

comprehensive documentation

Customer collaboration

OVER

contract negotiation

Responding to change

OVER

following a plan

“That is, while there is value in the items on the right, we value the items on the left more.”

Beck, K. et al., *Manifesto for Agile Software Development*, 2001. The authors place the declaration under a notice permitting it to be copied freely, provided it is copied in its entirety. It is reproduced here in full for that reason.

THE SENTENCE EVERYONE DROPS

The closing line is not a politeness. It is the load-bearing part. Each value is a **trade**, not an abolition: the items on the right still have value, and a team that reads only the left-hand column produces exactly the caricature that gave agile its bad name — no documents, no plan, no contract, and a project that cannot be handed to anyone.

So the useful way to read each line is as a pair of questions: **what does the trade buy me**, and **what do I still owe?**

The value	What the trade buys	What you still owe
Individuals and interactions over processes and tools	The clerk says the search screen confuses her. Two students go to the desk on Friday with the tablet. No ticket, no triage queue, no three-day delay.	A note in the repository the same day. An interaction that leaves no trace is a conversation, not engineering — the next student cannot inherit it.
Working software over comprehensive documentation	In week 6 the clerk types into a real registration screen rather than reading a forty-page specification. Her reaction <i>is</i> the requirement.	The lightweight requirements document of Session 9. The identifier format and the sync contract cannot be discovered by typing into a screen.
Customer collaboration over contract negotiation	In week 8 the nurse asks for a follow-up list. It goes into the backlog and something else moves down. Nobody argues about whether it was in scope.	One written non-negotiable. Whatever else is cut, <i>no record is lost in a power cut</i> ships in version 1.0. Collaboration without a fixed floor is drift.
Responding to change over following a plan	A paper prototype in week 5 shows that searching by the mother's name is essential. It enters sprint 3; card printing leaves.	A goal that does not move <i>inside</i> the sprint. Changing direction every day is not responsiveness; it is the absence of a decision.

The mistake to expect from yourselves

Every student team in this course has at some point justified skipping a test, a note or a requirements document by citing the Manifesto. None of the four values mentions tests, and the right column never says “worthless”. If your use of the Manifesto always happens to save you work, you are quoting half of it.

3 • The unit of work: the user story

Before we can run a sprint, we need something a sprint can consume. The requirements you wrote in earlier sessions look like this:

A REQUIREMENT STATEMENT — CORRECT, AND UNUSABLE FOR PLANNING

SR-1.1 The system shall allow a clerk to register a patient using the fields recorded on the existing paper card.

That sentence is fine for a contract and useless for a two-week plan. It has no owner, no size, no benefit, and the clerk cannot look at it and tell you whether it is worth building first. A user story fixes all four.

DEFINITION — USER STORY

A **user story** is one piece of work, written from the point of view of the person who wants it, small enough to finish inside a sprint and concrete enough for that person to judge when it is done. It follows a fixed three-clause template:

As a <role>, I want <capability>, so that <benefit>.

A story is a **promise of a conversation**, not a specification. The conversation happens when the story is planned; what the conversation produces is written underneath as **acceptance criteria**.

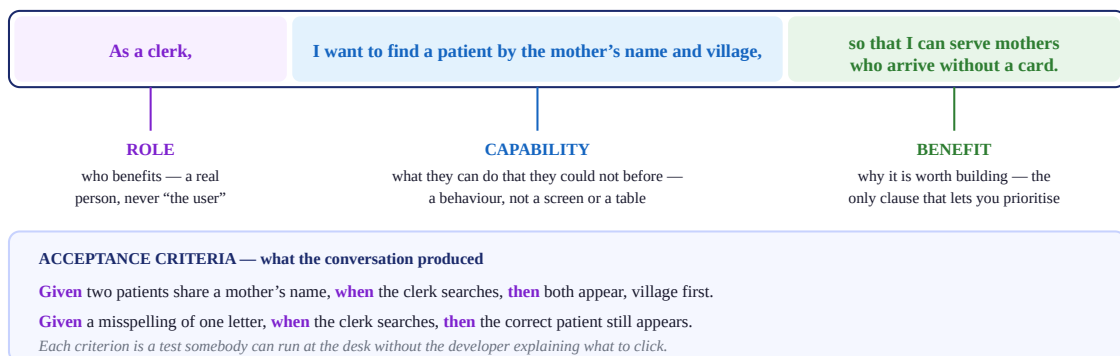


Figure 2. The three clauses and what each one is for. Students almost always write the first two and omit the third — and then cannot explain why one story should be built before another.

3.1 Testing a story: INVEST

Six letters, six quick questions. A story that fails one of them is not forbidden; it is a warning that planning will be painful.

I

Independent

Can it be built without waiting for another unbuilt story? If not, order the two.

N

Negotiable

Is there room to discuss *how*? A story that names a widget has already closed the conversation.

V

Valuable

Does the benefit clause name something a real person gains? “so that the database is normalised” fails.

E

Estimable

Do you know enough to size it? If not, the honest answer is a spike, not a bigger number.

S

Small

Can the six of you finish it inside one sprint, with room for the other stories? If not, split it.

T

Testable

Can you write Given/When/Then for it now? If you cannot, you do not yet know what it means.

APPLIED TO SANTE-BASSAM

One story repaired, one story worked in full

S-05 · first draft

“As a user, I want a better search, so that the system is easier to use.”

Fails **V** (“easier to use” is not a benefit anyone can weigh against another story), fails **T** (there is no test for “better”), and fails the role clause (“a user” is nobody). It is not a small story badly written — it is three different stories that have not been separated yet.

S-05 · repaired

As a **clerk**, I want to **find a patient by the mother’s name and village**, so that **I can serve mothers who arrive without a card**.

ACCEPTANCE CRITERIA · TRACES TO SR-1.1, SR-1.2

AC1 **Given** a mother’s name and a village, **when** the clerk searches, **then** every matching patient is listed, those in the named village first.

AC2 **Given** a name misspelled by one letter, **when** the clerk searches, **then** the correct patient still appears in the first five results.

AC3 **Given** the full register of 6,000 patients, **when** the clerk searches, **then** results appear in under two seconds on the desk laptop.

INVEST check. **I** — needs only the local store, finished in sprint 0. **N** — nothing here says how ranking works. **V** — mothers without cards are roughly a fifth of arrivals. **E** — comparable to work already done. **S** — fits a sprint alongside two others. **T** — AC1–AC3 are already three system tests.

3.2 Sizing: story points and velocity

Every team that estimates in hours discovers the same two things. Students are bad at estimating hours, and an hour from a tired student in week 12 is not an hour from a fresh one in week 5. So agile teams estimate *relative size* instead, and let arithmetic supply the rest.

DEFINITIONS — POINTS AND VELOCITY

A **story point** is a unit of relative size: effort, complexity and uncertainty rolled into one number. Points are meaningful only by comparison. Pick one small, well-understood story, call it **2**, and size everything else against it, using only the values **1 · 2 · 3 · 5 · 8 · 13**. The gaps widen deliberately: nobody can tell a 7 from an 8, and pretending otherwise wastes the meeting.

Velocity is the total number of points a team actually completed in a sprint — completed to the Definition of Done, not “nearly finished”. It is measured, never promised. After one or two sprints it becomes the only honest answer to “how much can you take on?”

Points	What it means for your six-student team	Example from SANTE-BASSAM
1-2	Understood, small, no surprises expected.	Show the patient’s village on the result row.
3	A day or so of pair work; one clear piece of behaviour.	S-06 — search result under two seconds.
5	Several parts, but all of them understood.	S-04 — register a patient with the paper-card fields.
8	Large, or containing something nobody has done before.	S-05 — search with ranking and misspellings.
13	Too big to plan. Split it, or spend a timeboxed spike first.	“Offline sync” before it was split into S-07, S-08, S-09.

Why not just say hours?

Because points are a property of the work and hours are a property of the week. Relative sizes stay true when exams arrive; hours do not. Velocity then converts points into calendar reality automatically — if the team completes fewer points during exam weeks, the plan absorbs it without anyone having to re-estimate a single story.

Six people will not finish 50% more than four

Your team is six, and sprint 0 measured 16 points. Do not scale that to 24 because you have more people. Adding people adds communication paths, not output: four people have six pairwise channels, six people have fifteen. Some of the extra capacity is spent on coordination, some on the second person now waiting to have their work reviewed. Plan with the number you measured, let sprint 1 measure it again, and adjust

from evidence. Friday's session shows what actually happens to a six-person team that starts more work than it finishes.

4 • Scrum: a framework for scheduling feedback

Scrum is the most widely used agile framework, and the most widely misunderstood. It is worth being blunt about what it is before listing its parts.

ONE RULE FOR THE WHOLE FRAMEWORK

Scrum schedules feedback. It says nothing about how to write code. Every one of its five events exists to create a fixed, unavoidable moment at which somebody inspects something: the review inspects the product, the retrospective inspects the process, the daily inspects the plan. That is the entire contribution. Teams that adopt Scrum and keep bad engineering habits get feedback on schedule about software they cannot safely change — and conclude that agile does not work. Friday's session supplies the missing half.

Scrum has eleven parts, and they come in three kinds. Take them one kind at a time, and for each ask the same two questions: **who owns it**, and **what does it inspect**?

4.1 The sprint — the container everything else sits in

DEFINITION — SPRINT

A **sprint** is a fixed period, one to four weeks, at the end of which there is an increment of working software. Three things are fixed the moment it begins: its **length**, its **goal**, and the fact that it **does not get extended**. If the work does not fit, the work goes back to the backlog — the sprint never moves. That single piece of rigidity is what makes velocity measurable and what makes the review date credible.

4.2 Three accountabilities — who is answerable for what

Product Owner

ONE PERSON, NOT A COMMITTEE

Answerable for the **value** of what gets built. Owns the ordering of the product backlog and has the final word on it. Says what and why; never how.

SANTE-BASSAM: one student, acting as proxy for the nurse. The nurse cannot attend planning, so someone orders the backlog daily on her behalf and answers to her at each fortnightly review, where she has the final say.

Scrum Master

A FACILITATOR, NOT A MANAGER

Answerable for the **process working**. Runs the events to time, removes blockers, and protects the sprint goal from mid-sprint additions. Has no authority to assign work.

SANTE-BASSAM: a second student, rotating each sprint. Rotation stops one person quietly becoming the manager. Five sprints and six of you means the rotation does not divide evenly — agree the order in sprint 1 and write it down, rather than discovering it in week 9.

Developers

EVERYONE WHO BUILDS IT

Answerable for **how** the work is done and for meeting the Definition of Done. They, not the Product Owner, decide how much fits in a sprint.

SANTE-BASSAM: all six of you, the Product Owner and Scrum Master included. Those two are **accountabilities held alongside development work, not jobs**: everyone on this team writes code and everyone tests, in every sprint. A six-person team that lets two people stop building is a four-person team with extra meetings.

4.3 Three artifacts — and the distinction that causes the most confusion

Product backlog

OWNED BY THE PRODUCT OWNER

The single **ordered** list of everything that might ever be built, expressed as user stories. It is never complete and never frozen: items are added, removed, re-ordered and re-sized at any time. Ordering is by value and risk, not by ease. Only the top of the list needs to be detailed.

SANTE-BASSAM: sixteen stories, S-01 to S-16, ordered against the risk register from Session 4.

Sprint backlog

OWNED BY THE DEVELOPERS

What the team **selected for this sprint** and how they intend to build it — three things together: the **sprint goal** (one sentence), the **stories chosen** from the top of the product backlog, and the **tasks** those stories were broken into. It is the team's own plan, and only the team changes it, daily, during the sprint.

SANTE-BASSAM: for sprint 1 — one goal sentence, three stories totalling sixteen points, and the tasks underneath them. Worked in full in §5.3.

Increment

OWNED BY THE DEVELOPERS

The working software at the end of the sprint — everything completed in this sprint plus everything completed in all previous ones, still working together. An item belongs to the increment only if it meets the **Definition of Done**.

SANTE-BASSAM: the version installed on the desk laptop at the review, which the clerk then keeps using until the next one.

Product backlog vs. sprint backlog — the difference in one line

The **product backlog** is everything that might be built, ordered by the Product Owner, and it outlives every sprint. The **sprint backlog** is the slice the Developers pulled off the top for the next two weeks, plus their plan for it, and it is thrown away when the sprint ends. Different contents, different owner, different lifetime. Confusing them is the single most common error in this unit's coursework.

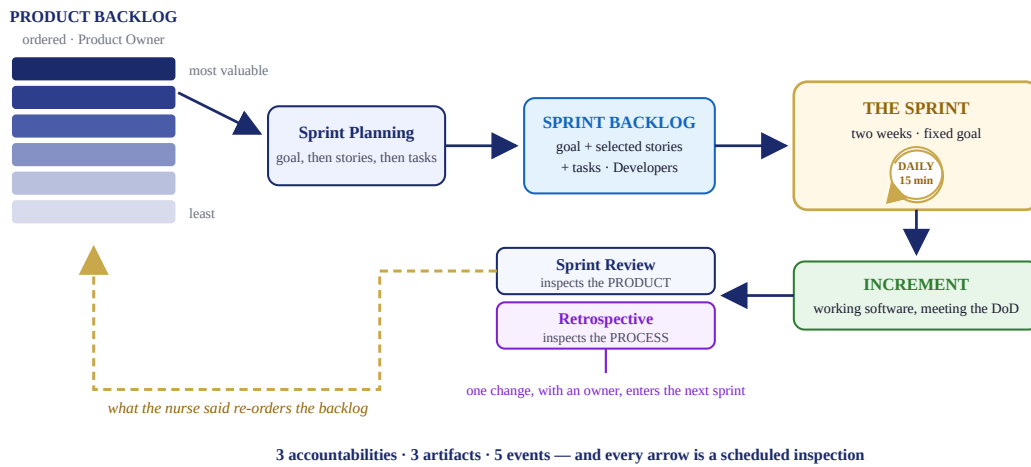


Figure 3. The loop. Read it left to right once, then follow the gold arrow back: the review is not the end of the process, it is the input to the next ordering of the product backlog. The increment is the only artifact the customer ever sees.

4.4 Five events — each one a scheduled inspection

Event	Timebox	What happens	What it inspects, and what comes out
The Sprint	2 weeks	The container for the other four. Begins the moment planning ends.	Nothing — it is the period itself. Out: an increment.
Sprint Planning	≤ 90 min	The Product Owner proposes a goal; the Developers pull stories off the top of the product backlog until they reach their velocity, then break them into tasks.	Inspects the product backlog. Out: the sprint backlog.
Daily Scrum	15 min, standing	Finished since yesterday · doing today · blocked by what. Nothing else is discussed here; problems get a separate conversation afterwards.	Inspects progress against the sprint goal. Out: today's plan, and a written blocker.
Sprint Review	≤ 60 min	The customer <i>uses</i> the increment. The team watches and writes; it does not demonstrate and it does not explain.	Inspects the product. Out: a re-ordered product backlog.

Retrospective

≤ 45 min

The team alone. What to keep, what to change — about how they work, not about what they built.

Inspects the process. Out: one change, with an owner, in the next sprint backlog.

4.5 The Definition of Done

DEFINITION — DEFINITION OF DONE

The team’s written, shared standard for what “finished” means. It applies to every story, it is agreed once and changed deliberately, and it lives in the repository where anyone can read it.

Work that does not meet it is not part of the increment, does not count toward velocity, and cannot be shown at the review.

WHY THIS IS THE ELEMENT THAT DECIDES WHETHER THE REST WORKS

Without a written Definition of Done, each story is finished to a different standard, so the points are not comparable, so velocity is a fiction, so planning is guesswork, so the review date is a hope. Every other part of Scrum depends on this one paragraph being real.

5 · SANTE-BASSAM: the whole framework, instantiated

Everything above has now been defined. This section does nothing new — it just fills in each definition for your team, so that you can see what the eleven elements look like when they belong to six students, one clerk, one community health worker and one nurse.

5.1 The calendar

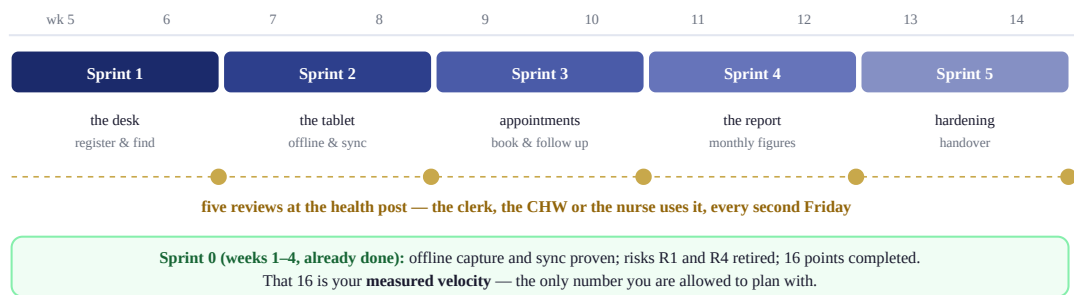


Figure 4. Ten weeks, five sprints, five dates on which a named person touches the software. The waterfall team of Session 3 reached its first such date in week 12. That gap is the whole argument.

5.2 The product backlog after sprint 0

Sixteen stories, ordered against the risk register from Session 4 rather than by ease of building. The table is a compression of the story template you met in §3: each row reads **as a** <role>, **I want** <capability>, **so that** <benefit>.

#	As a...	I want...	so that...	Traces	Pts	Sprint
S-01	team	a journaled local store on both devices	no record is lost when the power fails	SR-2.1	5	0 ✓
S-02	team	locally generated patient identifiers	two devices can register offline without collision	SR-2.1	3	0 ✓
S-03	clerk	the paper register migrated into the system	returning patients already exist on day one	UR-1	5	0 ✓
S-04	clerk	to register a new patient using the paper-card fields	I stop writing every patient down twice	SR-1.1	5	1
S-05	clerk	to find a patient by the mother's name and village	I can serve mothers who arrive without a card	SR-1.1, 1.2	8	1
S-06	clerk	search results in under two seconds	the queue at the desk does not grow	SR-1.3	3	1
S-07	CHW	to register a patient while the tablet is offline	I can enrol families during village outreach	SR-2.2	5	2
S-08	nurse	offline records to reach the desk in creation order	the register stays consistent	SR-2.2	5	2

S-09	nurse	records that fail to sync three times flagged for review	nothing vanishes silently	SR-2.3	3	2
S-10	clerk	to book an appointment on a date	the nurse can plan her day	UR-4 (new)	5	3
S-11	nurse	each visit recorded as attended, missed or rescheduled	the monthly report is true	SR-3.1	3	3
S-12	nurse	a list of patients due for follow-up this week	I can retire the private notebook I keep	UR-5 (new)	5	3
S-13	officer	a monthly report by village and outcome	I stop counting rows in the paper register	SR-3.1	5	4
S-14	officer	the report exported as a spreadsheet	I can check two villages by hand	SR-3.1	3	4
S-15	clerk	a warning when a registration resembles an existing patient	I do not create duplicate records	R7	5	4
S-16	clerk	to print a patient card	the patient leaves with something in hand	nice-to-have	3	—

Read the ordering, not the numbering. S-16 is last not because it is hard — it is one of the cheapest — but because nobody is harmed by its absence. S-05 is second despite being the most expensive story on the list, because a mother turned away at the desk is the failure the nurse described first.

5.3 The sprint backlog for sprint 1

This is the artifact students most often get wrong, so here it is in its three layers, in the order sprint planning produces them. **Goal first, then stories, then tasks** — never the other way round.

LAYER 1 THE SPRINT GOAL — PROPOSED BY THE PRODUCT OWNER

“On Friday 16 October, the clerk registers three new patients and finds three returning ones on the desk application, unaided.”

One sentence. It names a person, a date and something observable. It says nothing about screens, tables or code — if the team finds a cheaper way to achieve it, the goal still stands.

LAYER
2

THE STORIES PULLED FROM THE TOP OF THE PRODUCT BACKLOG — CHOSEN
BY THE DEVELOPERS

Story	Why it is in this sprint	Pts
S-04	Without registration there is nothing to find.	5
S-05	The goal sentence says “finds three returning ones”.	8
S-06	“Unaided” fails if the clerk waits ten seconds per search.	3
Total — equal to the velocity measured in sprint 0		16

Left out on purpose: S-07 (the tablet). It is next in the order, but adding it makes 21 points against a measured velocity of 16, and a sprint that overcommits does not fail gracefully — it fails at the review, in front of the clerk.

LAYER 3

THE TASKS — WRITTEN BY THE DEVELOPERS, CHANGED DAILY

Breaking down S-05, the largest story:

- Paper-prototype session with the clerk at the desk — **day 3, before any screen is built**
- Write the failing test for the ranking function (AC2, the misspelling case)
- Implement `rank()` until that test passes
- Build the prefix index over the 6,000 migrated records (needed for AC3)
- Build the search screen against the agreed prototype
- Script AC1 and AC3 as automated system tests

Notice the first and second items. The conversation with the clerk and the test both come *before* the screen. That ordering is not Scrum — Scrum is silent about it. It is the engineering half, and it is Friday’s session.

5.4 The team’s Definition of Done

DOD.MD — IN THE REPOSITORY, AGREED BY ALL SIX STUDENTS

A story is DONE when, and only when:

1. it is merged to main through a pull request reviewed by someone who did not w
2. every new function has unit tests, and the whole suite is green in CI on the
3. each acceptance criterion has been demonstrated on the target device
4. new fields are in the data dictionary; new interfaces are in the sync contrac
5. the traceability matrix is updated: story -> requirement -> tests
6. the nurse or the clerk can use it without a developer explaining what to clic

Six lines, and every one of them is a line some team has learned the hard way. Item 3 is the one that will bite you in sprint 2: “the target device” stops meaning the laptop on the desk and starts meaning a tablet in a village with no network.

SESSION 5 TAKEAWAY

Agile is what a team does when it can afford to be wrong cheaply. The Manifesto states four trades and, in the sentence everyone drops, refuses to abolish either side of them. The user story is the unit of work small enough to finish and concrete enough for a real person to judge; points size it and velocity converts points into calendar truth. Scrum then schedules the feedback — three accountabilities, three artifacts, five events — and gives your team one thing the waterfall team of Session 3 did not have: Friday 16 October, the day the clerk touches the software.

6 · In-class exercise · repair three stories

Ten minutes, on your own — then two minutes comparing with the person next to you. Each of these fails at least two INVEST letters. Rewrite it in the three-clause template and name the letters it failed. We take the first one together; the other two are your homework.

The three broken stories

1. “As a developer, I want to refactor the sync module, so that the code is cleaner.” — *together, in class*
2. “As a nurse, I want a dashboard with charts and filters and an export button, so that I have an overview.”
3. “As a user, I want the system to be fast.”

As a **<a named role from SANTE-BASSAM>**,
I want **<one capability>**,
so that **<a benefit that person would recognise>**.
Fails: **<letters>** — because **<one line>**

WORKED MODEL — A FOURTH BROKEN STORY

“As an admin, I want a settings page, so that things can be configured.”

Rewritten: As the **nurse**, I want to change the opening hours shown on the appointment screen, so that the clerk stops booking patients into the Friday vaccination clinic.

Fails: V and T — “things can be configured” names no beneficiary, so no test can be written and no ordering decision can be made.

Where sprint planning went

Planning a sprint is team work, not exercise work, so it belongs in the fifteen-minute project slot rather than here. Sprint 1's tasks are worked together at the start of Session 6, and sprint 2 is planned at the start of Session 7 once elicitation has given you something worth planning. What you practise alone today is the skill all of that rests on: writing one story somebody else can build and the nurse can judge.

7 · Homework · individual · due before Session 6 (Friday 25 September)

This one is **yours alone**, not the team's. It should take about forty-five minutes and none of it is hard. It exists so that each of the six of you can write a story unaided rather than watch somebody else do it, and it continues directly from §6 and from the project slot at the start of class.

TEMPLATE

Download **CSC4305_HW05_Template.docx** from **CSC_4305_SWE-Assignments**

— fill it in, do not delete the questions or the tables. **Rename** it

CSC4305_HW05_FirstName_LastName.docx. Commit it to **/homework/S05/** in

your own private repository (CSC_4305_SWE-Student-FirstName-LastName) — never to the assignments repository, and never to the team repository.

A .docx file. Not a PDF, not a Google Docs link, not **.pages** or **.odt**. Full instructions are in the *Read this first* box at the top of the template.

Q1 Your own sentence, against the template. Take the story your pair discussed in the project slot — S-01, S-02 or S-03 — and write it out in the three-clause template. Then one sentence: what did the template force you to say that your spoken answer had left out?

Almost everybody leaves out the benefit. Catching that in your own writing is the whole point of the question.

Q2 Repair the two remaining broken stories from §6 — the nurse's dashboard, and "the system should be fast". Template, plus the INVEST letters each one failed with a one-line reason.

The dashboard is not one story. Say how many you think it is, and name them.

Q3 Write one new story of your own. Any capability SANTE-BASSAM does not yet have, for any role *other than the clerk*. Add two or three acceptance criteria in Given/When/Then form, and a point estimate from the 1·2·3·5·8·13 scale with one line saying what you compared it against.

Compare it to a story whose size you already know. That comparison is what relative estimation is.

Q4 INVEST-check your own story. One line per letter. If it fails one, say so and keep it — an honest failure scores better here than a story bent to pass.

Q5 In your own words, two or three sentences: what is the difference between the product backlog and the sprint backlog? If your answer could have been written without reading §4.3, write it again.

Q6 **Reading.** Sommerville, *Software Engineering* 10th ed., §3.1–3.2, then Schwaber & Sutherland, *The Scrum Guide* (2020) — fourteen pages, read all of it. Bring one sentence to Friday naming something the Guide says that this lecture did not.

Two repositories, and the difference matters. Homework goes in *your own private repository*: it is individual work and it is marked individually. The backlog, the sprint backlog, the Definition of Done and the progression log go in the *team repository*: they are built and amended in the project slot at the start of each session, and nobody should be writing them alone the night before.

Key terms

cost of feedback

Agile Manifesto

user story

acceptance criteria

Given/When/Then

INVEST

story point

velocity

sprint

Product Owner

Scrum Master

product backlog

sprint backlog

sprint goal

increment

Definition of Done

Readings and references

- Sommerville, I. *Software Engineering*, 10th ed. — Chapter 3.
- Beck, K. et al. *Manifesto for Agile Software Development*, 2001 — the four values and the twelve principles.
- Schwaber, K. & Sutherland, J. *The Scrum Guide*, 2020.
- Cohn, M. *User Stories Applied*, 2004 — chapters 1–2 for INVEST and story splitting.